SELF-PACED SYSTEMS MASTERCLASS

GIT GOOD

The 365-Day SWE Playbook

```
$ git commit -m "fix(career): might take 1 year to complete" --no-verify
```

Written & Verified by me

2026 · First Edition

About the Author and This Book

About the Author

I'm Srajan Rai, a software engineer writing backend systems in C#/.NET who got deeply tired of current job market and levels of interviews that demand everything from theory to actual hands-on experience.

I got laid off and took some time off to react to this job famine, So after getting back from my vacation and preparing for 1 month 16 hrs a day, I got the job and wanted to create a self pace play book to help myself. I created this book to help myself and others like me. I wanted to understand the absolute everything: how CPU pipelines execute instructions, why cache line false sharing degrades performance, how lock-free primitives eliminate contention, and how state machines prevent split-brain anomalies under network partitions.

The standard was simple: if I couldn't explain the physical hardware mechanics, sketch the memory layout, trace the failure recovery path, and defend the trade-offs on a whiteboard under pressure, I didn't actually own the concept. This playbook is that roadmap.

About This Book

This is not a high-level cheat sheet, and it is not an academic textbook created by a human. It is an unapologetic, exhaustive **365-day, 19-volume systems curriculum** engineered to take me from any level to Staff-level systems design and architecture.

Every single module follows an unyielding **5-Layer Pedagogical Framework**

- **1. Physical Memory & Hardware Reality:** What physical RAM, CPU registers, and 64-byte cache lines are doing under the hood during execution.
- **2. Abstract Theory & Mathematical Invariants:** Formal complexity bounds (O, Ω, Θ), proof-of-correctness invariants, and state transition models.
- **3. Production-Grade Implementation:** Complete, unabridged source code with zero-allocation optimizations, error handling, and memory safety.
- **4. Elite Algorithmic & Real-World Scenarios:** Core interview archetypes, non-trivial edge cases, and high-frequency failure scenarios.
- **5. Verification, Synthesis & Examination:** Self-assessment checklists and multi-question quizzes with full technical rationale to audit knowledge gaps.

Table of Contents

Module 3: Scripting Languages & Enterprise Automation (Python & C# Core)

- **Day 24:** Python Advanced Syntax: Dunder Methods, Iterators, and Generators
- **Day 25:** Concurrency: Asyncio Event Loops, Coroutines & Threading vs. Multiprocessing
- **Day 26:** Reference Counting, Garbage Collection & Cython/Numba Acceleration
- **Day 27:** Python Typing, Packaging (pyproject.toml, Poetry) & Pytest Fixture Architectures
- **Day 28:** C# Environment Setup, .NET CLI, and the Roslyn Compiler Pipeline
- **Day 29:** C# Type System: Value Types, Reference Types, and Stack vs. Heap Memory Layouts
- **Day 30:** Nullable Reference Types, Safe Null Handling & Immutable Data Records
- **Day 31:** Object-Oriented Principles in C#: Classes, Structs, Polymorphism & Interface Design
- **Day 32:** Generics, Type Constraints, and Covariance/Contravariance Mechanics
- **Day 33:** Delegates, Events, Lambda Expressions & Expression Trees
- **Day 34:** LINQ Architecture: Deferred Execution, Expression Trees, and Custom Providers
- **Day 35:** Exception Handling Strategies, Custom Exception Hierarchies & Result Monads
- **Day 36:** Asynchronous Programming: Task, ValueTask, and Async Streams
- **Day 37:** Multithreading in C#: ThreadPool Tuning, Mutexes, Semaphores, and Monitors
- **Day 38:** .NET Memory Management: Garbage Collector Generations, Ephemeral/LOH heaps
- **Day 39:** Unsafe Code, Pointers, Memory Marshaling & Interop with Native Libraries (P/Invoke)
- **Day 40:** Zero-Allocation Programming: Span, ReadOnlySpan, and Memory<T> Manipulations
- **Day 41:** Reflection, Dynamic Code Execution, Attributes, and Source Generators
- **Day 42:** Argon2id Key Derivation, AES-256-GCM, and Secure Memory Zeroing
- **Day 43:** File I/O Streams, Binary Serialization, and High-Performance JSON Processing
- **Day 44:** Performance Profiling: xUnit, Moq, Fluent Assertions, and BenchmarkDotNet
- **Day 45:** DI Container Lifetimes, Service Registration, and Composition Roots
- **Day 46:** Pattern Matching, Advanced Switch Expressions, and Top-Level Statements
- **Day 47:** Concurrent Collections, Channels, and Asynchronous Pipeline Architectures

Module 4: Code Quality, Static Analysis & Architectural Patterns

- **Day 48:** SOLID Principles Deep Dive with Enterprise C# Implementation Examples
- **Day 49:** Cyclomatic Complexity, Cognitive Complexity, and Metrics Tracking

- **Day 50:** Code Smells, Refactoring Techniques & Legacy Code Salvaging
- **Day 51:** AST-Level Static Code Analysis & Custom Roslyn Analyzer Development
- **Day 52:** SonarQube Architecture: Server, Scanner, Quality Gates & CI/CD Pipeline Integration
- **Day 53:** Code Coverage Metrics: Line, Branch, Mutation Testing & Technical Debt Management
- **Day 54:** Design Patterns (Creational & Structural): Singleton, Factory, Adapter, Decorator
- **Day 55:** Design Patterns (Behavioral): Strategy, Observer, Command & State Patterns
- **Day 56:** Clean Architecture & Domain-Driven Design (DDD) Strategic/Tactical Patterns
- **Day 57:** Domain Modeling: Entities, Value Objects, Aggregates, and Domain Events
- **Day 58:** CQRS and Event Sourcing: Immutable Event Logs, Snapshots & Event Stores
- **Day 59:** Resilient Application Architecture: Circuit Breakers, Retries, and Rate Limiting (Polly)
- **Day 60:** Refactoring a 10-Year-Old Legacy Monolith: A Strangler Fig Case Study

Module 5: Relational Databases & Storage Engines

- **Day 61:** ACID Guarantees, Relational Algebra & Normalization
- **Day 62:** B-Tree and B+ Tree Physical Page Layouts & Disk Seeking Optimization
- **Day 63:** Execution Plans, Index Tuning, Join Algorithms, and Filter Optimization Forensics
- **Day 64:** Stored Procedures, Triggers, User-Defined Functions & Concurrency Control
- **Day 65:** Azure SQL Database Architecture, Service Tiers, Elastic Pools & Managed Instances
- **Day 66:** Entity Framework Core Internals: Change Tracking, SQL Translation & Interceptors
- **Day 67:** Database Migrations, Table Partitioning, Sharding & Multi-Tenant Scaling
- **Day 68:** Database Security: TDE & RLS & Auditing
- **Day 69:** Performance Tuning: Query Store, Extended Events, DMVs & Parameter Sniffing
- **Day 70:** Redis In-Memory Caching and NoSQL Document Stores
- **Day 71:** Time-Series Databases for Telemetry & Graph Databases (Neo4j / Cypher Traversal)
- **Day 72:** Database Clustering, High Availability, Failover Groups & Disaster Recovery
- **Day 73:** Data Warehousing Concepts: Star Schema, Snowflake Schema, and Columnar Storage
- **Day 74:** ETL vs. ELT Pipelines, Change Data Capture (CDC) & Data Ingestion Strategies
- **Day 75:** Transactions: Two-Phase Commit (2PC) and the Saga Pattern

Module 6: Web APIs, Protocols & Enterprise Integration

- **Day 76:** Web API Architectural Styles & ASP.NET Core Fundamentals
- **Day 77:** Minimal APIs, Content Negotiation, Serialization, and RFC 7807 Problem Details
- **Day 78:** API Versioning Strategies, Lifecycle Management, and Backward Compatibility
- **Day 79:** Model Validation, FluentValidation, and Exception Handling Middleware
- **Day 80:** OData Protocol: Query Options ($filter, $expand, $select) and Custom Providers
- **Day 81:** OpenAPI, Swagger, and Enterprise API Documentation Standards
- **Day 82:** Microsoft Kiota: Generating Type-Safe API Clients from OpenAPI Specifications
- **Day 83:** Resilient HTTP Integration: IHttpClientFactory, Typed Clients, and Polly Pipelines
- **Day 84:** gRPC Services: Protobuf Schema Design, Multiplexing, and Bidirectional Streaming
- **Day 85:** GraphQL APIs: Schema Stitching, Queries, Mutations, and HotChocolate Execution
- **Day 86:** API Gateway Pattern and Reverse Proxies with YARP (Yet Another Reverse Proxy)
- **Day 87:** API Security: Rate Limiting Policies, CORS, IP Filtering, and Headers Protection
- **Day 88:** WebSockets, SignalR Hubs, and Scale-Out Backplanes
- **Day 89:** Enterprise Integration Testing with TestContainers and Automated Test Harnesses
- **Day 90:** API Monitoring, Health Checks, and Telemetry Instrumentation
- **Day 91:** Hypermedia (HATEOAS) and RESTful Maturity Models
- **Day 92:** Webhooks, Delivery Guarantees, Idempotency Keys, and Cryptography
- **Day 93:** Microsoft Graph API Authentication, Scopes, and OAuth 2.0 Token Flows
- **Day 94:** Querying Users, Groups, and Organizational Data via Microsoft Graph
- **Day 95:** Managing SharePoint, OneDrive Files, and Outlook Calendars via Graph SDKs
- **Day 96:** High-Performance Batching, Delta Queries, and Graph Change Notifications
- **Day 97:** Enterprise Integration Patterns (EIP): Message Channels, Routers, and Transformers
- **Day 98:** Python/C# Interop Pipelines: Running Cross-Language Automation Frameworks
- **Day 99:** Model Context Protocol (MCP): Standardizing Data Sources and Tool Integrations for AI

Module 7: Microsoft Fabric & Big Data Engineering

- **Day 100:** MS Fabric Architecture: OneLake, Lakehouses, Warehouses, and Spark Compute
- **Day 101:** Data Engineering in Fabric: Notebooks, PySpark, and JVM Interop

- **Day 102:** ACID Transactions, Transaction Logs, and Time Travel Mechanics
- **Day 103:** Data Factory Pipelines, Activities, Transformations, and Copy Data Engine
- **Day 104:** Real-Time Intelligence in Fabric: Kusto Query Language (KQL) and Eventstreams
- **Day 105:** Power BI Semantic Models, DirectLake Mode, and Optimized DAX Queries
- **Day 106:** Data Governance with Microsoft Purview: Lineage, Classification, and Scans
- **Day 107:** Securing Fabric Workspaces, Object-Level Security, and Row-Level Security
- **Day 108:** Streaming Data Ingestion: Azure Event Hubs to Fabric Real-Time Analytics
- **Day 109:** Orchestrating Complex Data Workflows, Monitoring, and Error Handling
- **Day 110:** Data Quality, Profiling, Cleansing, and Automated Validation Frameworks
- **Day 111:** Building Medallion Architecture: Bronze (Raw), Silver (Cleaned), Gold (Curated) Layers
- **Day 112:** Optimizing Spark Jobs: Catalyst Optimizer, Shuffle Partitions, and Caching Strategies
- **Day 113:** Integrating Azure SQL Operational Data Stores with Fabric Lakehouse Sync
- **Day 114:** Managing Fabric Deployment Pipelines and Git Integration for Data Assets
- **Day 115:** Custom Data Connectors, API Ingestion, and Webhook Data Pipelines
- **Day 116:** Data Mesh Concepts: Domain-Oriented Decentralized Data Ownership
- **Day 117:** Capacity Units Management, Throttling, and Cost Optimization in Fabric
- **Day 118:** Machine Learning Workloads and Model Training within Fabric Notebooks
- **Day 119:** Disaster Recovery, Data Replication, and High Availability in Analytics Platforms
- **Day 120:** Enterprise Data Platform Architecture Design & Governance/li>

Module 8: Containers, K8s, Messaging & GitOps

- **Day 121:** Containerization: Linux Namespaces, Cgroups, and Docker Optimization
- **Day 122:** Docker Compose, Multi-Stage Builds, and Container Vulnerability Scanning
- **Day 123:** K8s Architecture: Control Plane, Kubelet, Pods, and ReplicaSets
- **Day 124:** K8s: CNI Plugins, Services, Ingress Controllers, and Network Policies
- **Day 125:** Persistent Storage in K8s: PVs, PVCs, StorageClasses, and Statefulness
- **Day 126:** Helm Package Manager: Chart Templating, Values Management, and Releases
- **Day 127:** Service Mesh Architecture: Istio/Linkerd Sidecars, mTLS, and Traffic Shifting
- **Day 128:** GitOps CD: ArgoCD, Declarative State Reconciliation, and Rollbacks
- **Day 129:** Kafka Architecture: Partitions, Brokers, and Zookeeper/KRaft Consensus

- **Day 130:** Kafka Producers, Consumers, Consumer Groups, and Offset Management
- **Day 131:** Microservices under Docker/K8s: RabbitMQ AMQP Patterns vs. Kafka
- **Day 132:** K8s Monitoring: Prometheus Metrics Collection and Grafana Dashboards
- **Day 133:** K8s Alerting, SLO/SLI Tracking, and Alertmanager Config
- **Day 134:** K8s Autoscaling: HHPA, VPA, and Cluster Autoscaler
- **Day 135:** Advanced K8s: Custom Resource Definitions (CRDs) and Custom Operators
- **Day 136:** Zero-Downtime Rolling Updates, Blue-Green Deployments, and Canary Releases
- **Day 137:** Chaos Engineering: Injecting Network Latency and Pod Failures with Chaos Mesh
- **Day 138:** Comprehensive Container and K8s Engineering/li>

Module 9: Enterprise Security, Authentication & Cryptography

- **Day 139:** Enterprise Security Principles, Zero Trust Architecture & Threat Modeling
- **Day 140:** OAuth 2.0 Framework, Grant Types, Token Flows & OpenID Connect (OIDC)
- **Day 141:** JSON Web Tokens (JWT): Cryptographic Signing, Verification, and Claims Validation
- **Day 142:** Microsoft Entra ID (Azure AD): Tenants, App Registrations, and Enterprise Applications
- **Day 143:** Implementing Authentication and Authorization Pipelines in ASP.NET Core
- **Day 144:** Role-Based Access Control (RBAC) and Attribute-Based Access Control (ABAC)
- **Day 145:** Securing Single Page Applications (SPAs), Mobile Apps, and Public Clients
- **Day 146:** API Security: PKCE, Token Expiration, Rotation, and Refresh Token Theft Protection
- **Day 147:** Secrets: Key Vault, Managed Identities, and Rotation Strategies
- **Day 148:** Cryptography: Symmetric (AES), Asymmetric (RSA/ECC) Encryption, and Hashing
- **Day 149:** Enterprise Identity: MFA, Conditional Access, and SSO Integration
- **Day 150:** Comprehensive Enterprise Security and Identity/li>

Module 10: Cloud Infrastructure, IaC & Serverless Computing

- **Day 151:** Cloud Fundamentals, Global Infrastructure, Regions, and Availability Zones
- **Day 152:** Azure Compute: VM Scale Sets, App Service Slots, and Serverless Functions
- **Day 153:** ACA: Dapr Integration and Microservices on Serverless Infrastructure
- **Day 154:** Azure Storage Accounts: Blob Tiers, Table Storage, Queues, and File Shares
- **Day 155:** Azure Networking: VNets, Subnets, and Network Security Groups (NSGs)

- **Day 156:** Traffic Management: Load Balancers, Application Gateways, and Azure Front Door
- **Day 157:** Infrastructure as Code (IaC): Bicep Language Fundamentals and ARM Templates
- **Day 158:** Terraform Foundations: State Management, Providers, and Modular Design
- **Day 159:** Managing Azure Infrastructure at Scale with Terraform Enterprise Workflows
- **Day 160:** Azure Observability: App Insights, Log Analytics, and KQL Diagnostics
- **Day 161:** Azure Cost Management, Budgets, Resource Tagging, and Cloud FinOps Governance
- **Day 162:** Preparing for Cloud Certifications (AZ-204 / AZ-400 architectural blueprints)
- **Day 163:** Azure Policy, Custom Initiatives, and Governance Compliance Blueprints
- **Day 164:** Hybrid Cloud Connectivity: ExpressRoute, VPN Gateways, and Private Endpoints
- **Day 165:** Automated Cloud Deployments via Azure DevOps and GitHub Actions CI/CD
- **Day 166:** Disaster Recovery, Geo-Redundancy, and Backup Architectures in Azure
- **Day 167:** Comprehensive Azure Platform and Cloud Architecture/li>

Module 11: Artificial Intelligence, LLMs, Vector DBs & Multi-Agent Systems

- **Day 168:** Artificial Intelligence Landscape: Transformer Architecture & Attention Mechanics
- **Day 169:** Vector DB: High-Dimensional Embeddings, Cosine Similarity, and Inner Products
- **Day 170:** Anthropic Certifications Framework & Advanced Prompt Masterclass
- **Day 171:** RAG: Document Chunking, Overlap Strategies, and Hybrid Search
- **Day 172:** Advanced RAG: Semantic Reranking, Cross-Encoders, and Context Compression
- **Day 173:** Agentic AI Frameworks: Autonomous Task Planning, Reflection, and Tool Utilization
- **Day 174:** Semantic Kernel: Plugins, Planners, and Memory Connectors in C# and Python
- **Day 175:** AutoGen Framework: Multi-Agent Conversations and Collaborative Problem Solving
- **Day 176:** Python Agentic AI: LangChain, Tool Calling, and Text-to-SQL Autonomous Agents
- **Day 177:** Reinforcement Learning : Markov Decision Processes, Value Functions, and PPO
- **Day 178:** GraphRAG: Entity Extraction, Cypher Traversal, and Integrating Graph DB with LLM
- **Day 179:** Building Multi-Agent Systems with the ADK
- **Day 180:** Human-in-the-Loop Triage, Approval Workflows & Agentic State Management
- **Day 181:** Local LLM Runtime Deployment: Running Models with Ollama, vLLM, and Llama.cpp
- **Day 182:** Fine-Tuning Open Source LLMs: Low-Rank Adaptation (LoRA) and QLoRA Techniques
- **Day 183:** Ragas, BLEU, ROUGE, and LLM-as-a-Judge Validation

- **Day 184:** AI Guardrails: Content Moderation, Injection Defense, and PII Masking
- **Day 185:** Multimodal AI Architectures: Processing Images, Audio, and Video Streams
- **Day 186:** Optimizing Inference Costs, KV-Caching, and Latency in Production AI Deployments
- **Day 187:** Enterprise AI: Combining Semantic Kernel, Microservices, and Local LLMs
- **Day 188:** Fine-Tuning Pipeline Implementation with Python, PyTorch, and Hugging Face
- **Day 189:** Building Domain-Specific Knowledge Assistants with Vector DBs and RAG
- **Day 190:** Building Model Context Protocol (MCP) Servers in Python for Standardized AI Tooling
- **Day 191:** Comprehensive Multi-Agent AI and RAG Engineering/li>

Module 12: Client UI (.NET MAUI & Blazor)

- **Day 192:** Client UI Architecture: Introduction to .NET MAUI for Cross-Platform Apps
- **Day 193:** XAML Fundamentals: Layouts, Data Binding, Styles, and Triggers in MAUI
- **Day 194:** MVVM Pattern MAUI: Commands, Property Notifications, and Navigation Stacks
- **Day 195:** Blazor Architecture: WebAssembly (WASM) vs. SSR Execution Models
- **Day 196:** Blazor Components, Data Binding, Lifecycle Methods, and Event Handling
- **Day 197:** Native Device Feature Integration in .NET MAUI

Module 13: Core Data Structures & Algorithms (DSA)

- **Day 198:** Asymptotic Analysis: Big-O, Big-Omega, and Big-Theta Mathematical Proofs
- **Day 199:** Arrays, Strings, and Dynamic Array Memory Allocation Mechanics
- **Day 200:** Linked Lists: Singly, Doubly, and Circular Pointer Manipulations
- **Day 201:** Stacks, Queues, and Monotonic Stack/Queue Algorithmic Patterns
- **Day 202:** Hash Tables: Collision Resolution, Open Addressing, and Chaining Internals
- **Day 203:** Recursion, Backtracking, and Divide-and-Conquer Optimization Paradigms
- **Day 204:** Sorting Algorithms: Quicksort, Mergesort, Heapsort, and Radix Sort Math
- **Day 205:** Binary Search Mechanics and Advanced Two-Pointer Techniques
- **Day 206:** Binary Trees, Binary Search Trees, and AVL / Red-Black Tree Balancing
- **Day 207:** Heaps and Priority Queues: Implementation and Pattern Applications
- **Day 208:** Graph Representations: Adjacency Lists, Matrices, BFS, and DFS Traversals
- **Day 209:** Shortest Path Algorithms: Dijkstra, Bellman-Ford, and Floyd-Warshall

- **Day 210:** Minimum Spanning Trees: Kruskal's and Prim's Algorithm Implementations
- **Day 211:** Dynamic Programming: Memoization vs. Tabulation (1D and 2D DP)
- **Day 212:** Advanced Dynamic Programming: Bitmasking, Digit DP, and Interval DP
- **Day 213:** Greedy Algorithms and Interval Scheduling Proofs
- **Day 214:** Tries (Prefix Trees) and String Matching Algorithms (KMP, Rabin-Karp)
- **Day 215:** Disjoint Set Union (DSU) / Union-Find Data Structure with Path Compression
- **Day 216:** Sliding Window and Prefix Sum Masterclass
- **Day 217:** Memory Optimization and Layout Strategies in Data Structures
- **Day 218:** Concurrency-Safe Data Structures in Practice
- **Day 219:** Comprehensive Tier-1 DSA and Algorithmic Mastery/li>

Module 14: Applied FAANG Execution & LeetCode Hard Patterns

- **Day 220:** Advanced Array Manipulation & Matrix Transformations
- **Day 221:** Zero-Allocation Sliding Window & Two-Pointer Execution
- **Day 222:** Tree Architectures & Complex Graph Traversals
- **Day 223:** Backtracking & 2D Dynamic Programming Grinds
- **Day 224:** Specialized Structures (Tries, Segment Trees, DSU)
- **Day 225:** Custom LFU/LRU Cache Implementations under Concurrency
- **Day 226:** Multi-threading & Concurrency-Safe LeetCode Hards
- **Day 227:** Algorithmic Optimization: Mathematical Number Theory & Combinatorics
- **Day 228:** Mock Technical Interview Execution & Problem-Solving Frameworks
- **Day 229:** Comprehensive Applied FAANG Execution/li>

Module 15: Low-Level (LLD) & High-Level System Design (HLD)

- **Day 230:** Low-Level Design (LLD): Principles of Object-Oriented Modeling & Domain Invariants
- **Day 231:** Designing Systems: Parking Lot, Elevator Controller Movie Ticket Booking
- **Day 232:** Designing Tools: Splitwise Ledger, Multi-player Chess, and Rate Limiters
- **Day 233:** Design Patterns in Interviews: Factory, Strategy, and Observer Applied to LLD
- **Day 234:** High-Level System Design (HLD): Scalability, Availability, and Latency Metrics
- **Day 235:** Load Balancing Algorithms: L4 vs. L7, Round Robin, and Consistent Hashing

- **Day 236:** Caching Strategies: Cache-Aside, Write-Through, Invalidation, and Eviction Policies
- **Day 237:** Content Delivery Networks (CDNs) and Edge Computing Architecture
- **Day 238:** Database Scaling: Partitioning, Sharding, Replication, and Joins
- **Day 239:** Message Queues and Event Streaming: Event-Driven Architectures in HLD
- **Day 240:** Consensus: Paxos, Raft, and Leader Election Protocols
- **Day 241:** Transactions: Two-Phase Commit (2PC) and Saga Patterns in HLD
- **Day 242:** Designing Rate Limiters and Unique ID Generators (Snowflake ID)
- **Day 243:** Designing Large-Scale Systems: URL Shortener, Pastebin, and Web Crawler
- **Day 244:** Designing Feed Systems: Twitter / Instagram Timeline Architecture
- **Day 245:** Design RT Collab Platforms: Google Docs or Chat Systems
- **Day 246:** Designing Video Streaming Services: Netflix / YouTube Ingestion & CDN Architecture
- **Day 247:** Designing Ride-Sharing Systems: Uber / Lyft Geospatial Indexing (H3/Quadkey)
- **Day 248:** System Design Interview Framework: Requirements, Estimation, and Trade-offs
- **Day 249:** System Design Interview: Vocabulary, Trade-off Articulation, and Architectural

Module 16: Performance Profiling, Observability & Compilers

- **Day 250:** Performance Profiling: Identifying CPU Bottlenecks and Hot Paths in .NET
- **Day 251:** Memory Profiling: Diagnose Memory Leaks, GC Pressure, and Object Allocations
- **Day 252:** Live Debugging: Attaching Debuggers to Production Processes
- **Day 253:** Analyzing Minidumps, Thread Starvation, and OOM Killer Events
- **Day 254:** Application Observability: OpenTelemetry Tracing, Metrics, and Log Correlation
- **Day 255:** Telemetry: Jaeger and Prometheus Integration for Trace Propagation
- **Day 256:** Compiler Internals: Roslyn Syntax Trees, Semantic Models, and Compilation Pipelines
- **Day 257:** Writing Custom Roslyn Analyzers and Code Fix Providers
- **Day 258:** Just-In-Time (JIT) Compilation and Native AOT Compilation Mechanics in .NET
- **Day 259:** Wireshark Mastery, TCP Streams, and TLS Handshake Inspection
- **Day 260:** Database Query Forensics: Lock Escalation Debugging and Deadlock Forensics
- **Day 261:** Comprehensive Profiling, Observability & Compilers/li>

Module 17: Advanced Systems Engineering, Kernels & Storage Internals

- **Day 262:** Zero-Copy Serialization & Unmanaged Memory Layouts
- **Day 263:** Lock-Free LRU/LFU Caches & Memory Management
- **Day 264:** Futexes, Reader-Writer Locks, and Memory Barriers
- **Day 265:** Production Deadlock Engineering: Wait-For Graphs and Cycle Detection Algorithms
- **Day 266:** Database Storage Internals: B+ Tree Byte Layouts and Page Split Mechanics
- **Day 267:** Write-Ahead Logging (WAL) & ARIES Crash Recovery Architecture
- **Day 268:** Concurrency Control & Isolation: Two-Phase Locking (2PL) and MVCC Math
- **Day 269:** Kernel Bypass Architecture: DPDK, Solarflare EF_VI & Raw NIC Packet Ingress
- **Day 270:** Mechanical Sympathy & CPU Caches: False Sharing Mitigation & CPU Pinning
- **Day 271:** The LMAX Disruptor Pattern: Lock-Free Ring Buffers & Memory Barriers
- **Day 272:** Zero-Allocation Message Parsing: Parsing FIX Protocol without GC Triggers
- **Day 273:** TCP/IP Stack: Sliding Windows, ACK Storms & Congestion Control
- **Day 274:** Lexical Analysis: Building State-Machine Tokenizers without RegEx
- **Day 275:** Parsing & Abstract Syntax Trees: Recursive Descent Parsers & Operator Precedence
- **Day 276:** Semantic Analysis & Bytecode Emitters: Symbol Tables, Scoping & Stack-Based VMs
- **Day 277:** HTTP/3 & QUIC Protocol Internals: UDP Multiplexing & Connection Migration Math
- **Day 278:** WebRTC Deep Dive: ICE Candidates, STUN/TURN Servers & NAT Traversal Math
- **Day 279:** CLR Decompilation & MSIL Injection: Reading Raw IL & Obfuscation Bypass
- **Day 280:** Buffer Overflows & Return-Oriented Programming (ROP) on x64
- **Day 281:** Anti-Debugging & DRM: Sandboxing Detection, Timing Checks & Breakpoint Evasion
- **Day 282:** TinyML & MC Constraints: Quantizing Neural Networks FP32 -> INT8/INT4
- **Day 283:** ES & IoT: Interrupt Service Routines (ISRs), GPIO Registers & MQTT Binary Formatting
- **Day 284:** eBPF & Linux Kernel Networking: XDP Packet Filtering, Kprobes & Ring Buffers
- **Day 285:** Consensus Engineering: Building the Raft Protocol from Scratch
- **Day 286:** Vector Search Engine Internals: IVF, Product Quantization (PQ), HNSW & SIMD Math
- **Day 287:** RT Stream Processing: Event-Time Semantics, Windowing & Exactly-Once
- **Day 288:** Formal Verification & TLA+: Modeling of Systems & Proofs
- **Day 289:** Envoy Proxy Internals & xDS API: Building Custom Data Planes & L7 Traffic Routing

- **Day 290:** Legacy Codebase: Navigating 10yo Codebases & Strangler Fig Patterns
- **Day 291:** Feature Flags & Progressive Delivery: & Rollout Architectures
- **Day 292:** CI/CD Optimization: Docker Layer Caching, Test Parallelization & Flaky Test
- **Day 293:** Zero-Downtime Secret Rotation & IaC Drift Detection
- **Day 294:** Comprehensive Advanced Systems Engineering/li>

Module 18: DevOps, Enterprise Operations & Leadership

- **Day 295:** DevOps Culture, Continuous Delivery Pipelines, and Shift-Left Security
- **Day 296:** Infrastructure as Code (IaC) Advanced Strategies and Immutable Infrastructure
- **Day 297:** GitOps Advanced Workflows with ArgoCD and Automated Rollbacks
- **Day 298:** Disaster Recovery, Chaos Engineering, and Resilience Testing
- **Day 299:** Intent-Driven Development & Vibe Coding Methodologies
- **Day 300:** Intent-Based CI/CD: Integrating LLM Code Generation with GitHub Actions
- **Day 301:** Enterprise Architecture Governance and Compliance Frameworks
- **Day 302:** Staff+ Engineering Leadership: Technical Strategy, Mentorship, and Influence
- **Day 303:** Managing Stakeholders, Technical Debt, and Engineering Velocity
- **Day 304:** Cross-Team Collaboration and Large-Scale Migration Planning
- **Day 305:** Engineering Culture: Code Quality Enforcement and Automated Governance
- **Day 306:** Emerging Technology Trends: Quantum Computing, Edge AI, and WebAssembly
- **Day 307:** Open Source Contribution, Licensing, and Community Leadership
- **Day 308:** Enterprise Budgeting, Cloud Cost Optimization, and FinOps Architecture
- **Day 309:** Comprehensive DevOps and Enterprise Operations/li>

Module 19: Additional Engineering Projects

- **Day 310:** Advanced File Systems Architecture
- **Day 311:** WebAssembly & WASI: Stack-Machine Bytecode & Sandboxing Mechanics
- **Day 312:** Browser Architecture: Rendering Loops, Layout Trees & DevTools Internals
- **Day 313:** Lock Managers: Redlock and ZooKeeper-style Ephemeral Nodes
- **Day 314:** Performance Rate Limiting: Token Bucket and Leaky Bucket Algorithms
- **Day 315:** Tracing Collector: Building an OpenTelemetry Collector Ingress

- **Day 316:** Custom SQL Query Engine: Implementing Lexer, Parser, and Volley-Execution
- **Day 317:** Key-Value Store: Building LSM-Trees and MemTables
- **Day 318:** Network Packet Sniffer: Building a Raw Socket Packet Analyzer in C# / C++
- **Day 319:** Custom Message Broker: AMQP-style Exchange, Queues, and Routing Keys
- **Day 320:** Job Scheduler: Cron, Locking, and Fault-Tolerant Workers
- **Day 321:**Static File Server: Zero-Copy sendfile Implementation in User-Space
- **Day 322:** Custom Memory Allocator: Implementing Slab Allocation and Arena Memory Pools
- **Day 323:** Graph Processing: Pregel Vertex-Centric Processing Model Implementation
- **Day 324:** RTCS Engine: Scaling WebSockets Concurrent Connections via Epoll/IOCP
- **Day 325:** Custom JSON Parser: 0 Allocation Lexical Tokenizer and AST Builder
- **Day 326:** Consensus Simulator: Deterministic Network Partition Test Harness
- **Day 327:** SIMD Library: Optimizing Matrix Operations via AVX2/AVX-512 Intrinsics
- **Day 328:** Custom Load Balancer: Round-Robin and Least-Connections Routing
- **Day 329:** Metrics Aggregator: Building a High-Throughput Time-Series Ingestion Pipeline
- **Day 330:** Interpreter: Lexer, Parser, Stack Machine & Garbage Collector
- **Day 331:** Cache Invalidation Service: Consistent Hashing Ring with Virtual Nodes
- **Day 332:** High-Throughput Event Sourcing Engine: Append-Only Log with Zero-Copy Indexing
- **Day 333:** Blob Storage Gateway: Chunking, Erasure Coding, and S3-Compatible API Wrapping
- **Day 334:** Custom APM Agent: Intercepting Method Calls and Emitting Traces
- **Day 335:** Ledger / Blockchain Simulator: Merkle Trees, Proof-of-Work, and Block Validation
- **Day 336:** Spatial Indexing Engine: R-Trees and Quadtree Coordinate Calculations
- **Day 337:** Health-Checking Heartbeats and Dynamic Routing Tables
- **Day 338:** Lock-Free Queue: Multi-Producer Multi-Consumer (MPMC) Ring Buffer
- **Day 339:** Custom Web Framework: Building Routing Tables, Middleware, and Action Invokers
- **Day 340:** Rate Limiting Gateway: Redis-Backed Sliding Window Counter
- **Day 341:** CSV Parser: SIMD-Accelerated Streaming Parser for GB Datasets
- **Day 342:** Custom DI Container: Lifetime Scopes, Reflection, and Factory Binding
- **Day 343:** Circuit Breaker Engine: State Machine Transitions and Fallback Execution
- **Day 344:** Config Server: Hot-Reloading Key-Value Stores/ Watcher Notifications

- **Day 345:** Password Manager & Cryptographic Vault
- **Day 346:** Custom Chrome Extension Engine (Manifest V3 & Background Workers)
- **Day 347:** Real-Time Todo & State Synchronization App
- **Day 348:** React Architectural Deep Dive & Custom Concurrent Renderer
- **Day 349:** Fraud Detection: Scale AI Detection & FinCrime (AML) Rules Engine
- **Day 350:** Architecture Design, Multi-Region HA, Failover Topologies & Domain Modeling
- **Day 351:** Domain Implementation, Event-Sourced Storage, DB Tuning & Exe Forensics
- **Day 352:** Infrastructure Provisioning, K8s GitOps, CI/CD Pipelines & Canary Rollouts
- **Day 353:** Multi-Agent AI Integration, RAG & Semantic Kernel
- **Day 354:** Observability Instrumentation & OpenTelemetry Spans
- **Day 355:** Security Hardening, Zero Trust & OAuth2 Enforcement
- **Day 356:** Load Testing, Chaos Engineering, Resilience Validation, Core Dumps & Incident Triage
- **Day 357:** CN Core & Internet Plumbing: DNS, BGP, Router Packet Switching & TCP/IP Flow Control
- **Day 358:** OS: Thread vs. Process Address Spaces, Head Pointers & Concurrency Invariants
- **Day 359:** Bit Manipulation: Arithmetic, Bitmasking Tricks & LeetCode Pattern Taxonomies
- **Day 360:** Concurrent Caches, Sliding/Fixed TTLs, Async DB Fetching & Graceful Shutdown
- **Day 361:** Multi-Region HA, CI/CD Canary Rollouts & Event-Sourced Storage
- **Day 362:** Multi-Agent AI Integration, OTel Observability & 0-Trust Security
- **Day 363:** Chaos Engineering, Core Debugging & DB Execution Forensics
- **Day 364:** Go-Live: Final Security Audit, Cloud FinOps, Post-Mortems & Retrospective
- **Day 365:** The Ultimate Engineering Masterclass Retrospective, System Go-Live & Graduation

How to Use This PDF

This book is built by an AI, though annotated and content farmed by me.

The Whiteboard Pages

Every even-numbered page is a pure, solid white workspace. Use it to trace memory addresses, sketch failure state machines (e.g., worker failover, split-brain quorums), and write down your spoken whiteboard answers before checking the technical keys.

Color Legend

- **Indigo, Key Idea:** The foundational hardware contract, invariant, or architectural pattern.
- **Orange, Common Misconception:** Latent performance traps, race conditions, memory leaks, and flawed architectural assumptions.
- **Pink, Interview Angle:** How this exact mechanism is tested during FAANG, HFT, and senior backend technical rounds.
- **Green, Tip / Interview-Ready Answer:** Production takeaways, zero-allocation micro-optimizations, and crisp whiteboard phrasing.
- **Violet, Quiz & Verification:** Deep-dive self-assessment quizzes with comprehensive multiple-choice, writing prompts, and answer keys.

How to Take Notes

The whiteboards are not for rewriting what is printed. They are for synthesizing mechanics.

Restate the Physical Invariant

If a section explains CPU cache line false sharing or Raft log matching, sketch the multi-core or cluster topology yourself. Trace where the contention or quorum partition occurs.

Mark Failure Modes & Gaps

Whenever an invariant or edge case feels non-obvious (e.g., a node failing mid-transaction with an uncommitted write), place a prominent `?` on the whiteboard page. Re-verify the step-by-step state transitions.

Cross-Reference Across Volumes

When studying database storage engines in Volume X, cross-reference back to B-Tree cache line alignment in Volume II and asynchronous I/O in Volume VI. Senior systems engineering connects low-level primitives to high-level systems.

Rehearse Spoken Explanations

Answer the Pink Interview boxes out loud without looking at notes. Write your exact spoken explanation on the whiteboard page. If you cannot articulate the trade-offs cleanly out loud, you do not fully own the concept yet.